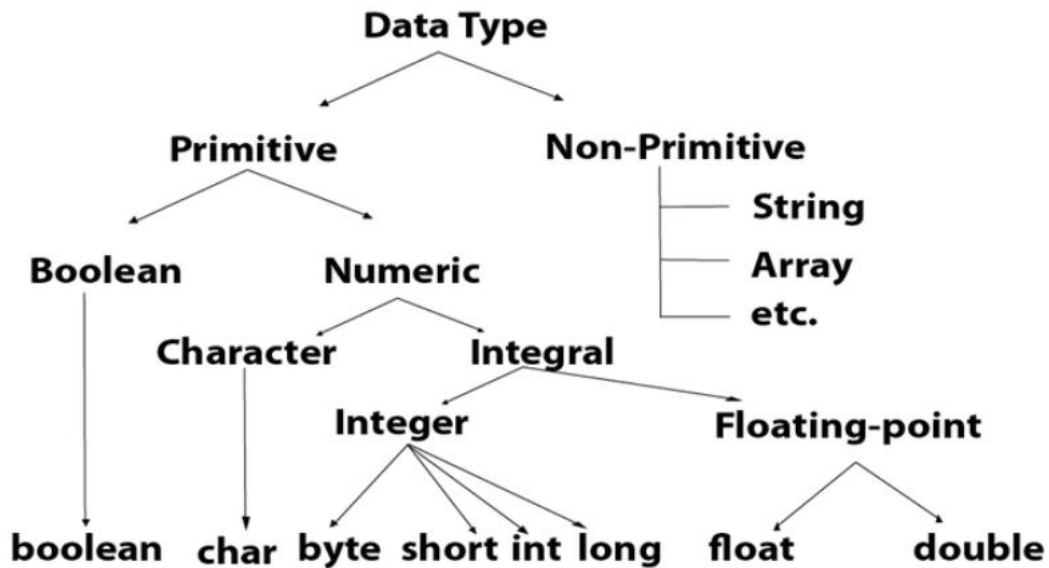


Java Data Types

Java data types define the type of data a variable can store in a program.



Data Type	Default Value	Default size	Example	Range
boolean	false	1 bit	Boolean one = false	true and false
char	'\u0000'	2 byte	char ch = 'A'	'\u0000' (or 0) to '\uffff' (or 65,535) (or) 0 to 65,536
byte	0	1 byte	byte a = 10	-128 to 127
Short	0	2 byte	short s = 10000	-32,768 to 32,767
int	0	4 byte	int a = 100000	-2,147,483,648 to 2,147,483,647
long	0L	8 byte	long a = 100000L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	0.0f	4 byte	float f1 = 234.5f	1.4e-045 to 3.4e+038
double	0.0d	8 byte	double d1 = 12.3	4.9e-324 to 1.8e+308

Primitive Data Types

Primitive data types are predefined by Java and are used to store simple values.

Data Type	Description	Example
byte	Small integer	byte age = 20;
short	Short integer	short year = 2026;
int	Integer	int marks = 95;
long	Large integer	long population = 8000000000L;
float	Single-precision decimal	float price = 99.5f;
double	Double-precision decimal	double pi = 3.14159;
char	Single Unicode character	char grade = 'A';
boolean	Logical value	boolean passed = true;

```
public class Main {  
    public static void main(String[] args) {  
        byte age = 25;  
        int salary = 50000;  
        System.out.println("Age = " + age);  
        System.out.println("Salary = " + salary);  
    }  
}
```

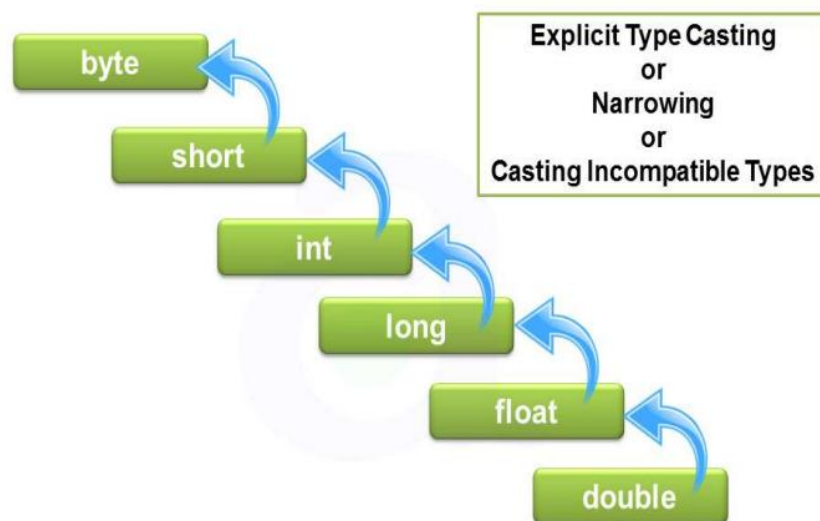
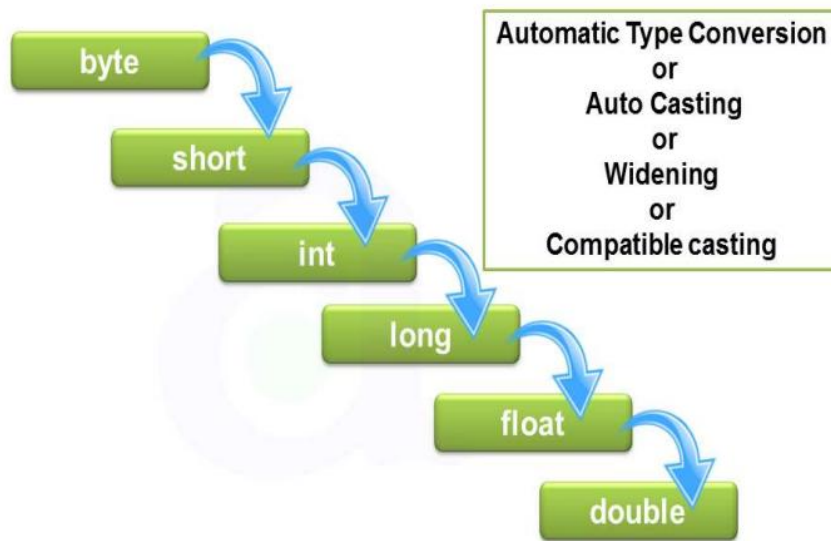
Age = 25
Salary = 50000

Overflow

```
public class Main {  
    public static void main(String[] args) {  
        byte b = 127;  
        b++;  
        System.out.println(b);  
    }  
}
```

-128

Type Conversion and Casting



Widening (Automatic)

```
public class Main {  
    public static void main(String[] args) {  
        byte b = 100;  
        int i = b; // Automatic conversion  
        System.out.println(i);  
    }  
}
```

100

Narrowing (Manual)

```
public class Main {
    public static void main(String[] args) {
        int i = 100;
        byte b = (byte)i;
        System.out.println(b);
    }
}
```

100

Arithmetic with byte: Arithmetic on byte, short, and char always produces an int.

```
public class Main {
    public static void main(String[] args) {

        byte a = 10;
        byte b = 20;

        // byte c = a + b; // Compilation Error (int + int)

        int c = a + b;

        //Type Promotion means Java automatically converts smaller data types to
        //a larger data type before performing an operation.

        System.out.println(c);
    }
}
```

30

Use Explicit Casting

```
public class Main {
    public static void main(String[] args) {
```

```
byte a = 10;
byte b = 20;

byte c = (byte)(a + b);

System.out.println(c);
}
```

30

(OR)

```
public class Main {
    public static void main(String[] args) {

        byte a = 10;
        byte b = 20;

        int c = a + b;

        System.out.println(c);
    }
}
```

30

```
public class Main {
    public static void main(String[] args) {
        char ch1 = 'A';
        char ch2 = 'B';
        //char ch3 = ch1 + ch2; // Error
        int result = ch1 + ch2;

        System.out.println(result);
    }
}
```

131

Be Careful with Overflow

```
public class Main {
    public static void main(String[] args) {

        byte a = 100;
        byte b = 100;

        byte c = (byte)(a + b);

        System.out.println(c);
    }
}
```

-56

```
// Demonstrate char data type.
class CharDemo {
    public static void main(String args[]) {
        char ch1, ch2;
        ch1 = 88; // code for X
        ch2 = 'Y';
        System.out.print("ch1 and ch2: ");
        System.out.println(ch1 + " " + ch2);
    }
}

output:
ch1 and ch2: X Y
```

```
public class Main {
    public static void main(String[] args) {
        // Constant assignments (Valid)
        byte b = 100;
        short s = 30000;
        char ch = 88;    // Unicode 88 = 'X'
        int i = 50000;
        long l = 100;

        System.out.println("byte = " + b);
    }
}
```

```

System.out.println("short = " + s);
System.out.println("char = " + ch);
System.out.println("int = " + i);
System.out.println("long = " + l);

// Widening conversions (Valid)
byte b1 = 10;

short s1 = b1; // byte -> short
int i1 = s1;    // short -> int
long l1 = i1;   // int -> long

System.out.println("\nWidening Conversions:");
System.out.println("short = " + s1);
System.out.println("int = " + i1);
System.out.println("long = " + l1);

// Narrowing conversions
int x = 100;

// byte b2 = x;    // ERROR
// short s2 = x;   // ERROR
// char ch2 = x;   // ERROR

// Explicit casting
byte b2 = (byte) x;
short s2 = (short) x;
char ch2 = (char) x;

System.out.println("\nAfter Explicit Casting:");
System.out.println("byte = " + b2);
System.out.println("short = " + s2);
System.out.println("char = " + ch2); // 'd' because Unicode 100 = 'd'
}
}

```

```

public class Main {
    public static void main(String[] args) {

        // float initialization
        float f1 = 10;    // int -> float (automatic widening)
        float f2 = 10.5f; // Correct (float literal)
        float f3 = (float)10.5; // Correct (casting)
    }
}

```

```

// double initialization
double d1 = 10;    // int -> double (automatic widening)
double d2 = 10.5;  // Correct (double is default)
double d3 = 10.5f; // float -> double (automatic widening)

System.out.println("f1 = " + f1);
System.out.println("f2 = " + f2);
System.out.println("f3 = " + f3);

System.out.println("d1 = " + d1);
System.out.println("d2 = " + d2);
System.out.println("d3 = " + d3);

// Invalid Examples
// float f4 = 10.5; // Compilation Error
// float f5 = d2;   // Compilation Error (double -> float)
}
}

f1 = 10.0
f2 = 10.5
f3 = 10.5
d1 = 10.0
d2 = 10.5
d3 = 10.5

```

Boolean

```

public class Main {

    // Instance variable (default value)
    boolean defaultValue;

    public static void main(String[] args) {

        // Explicit initialization
        boolean b1 = true;
        boolean b2 = false;

        // Boolean from a comparison
        boolean b3 = (10 > 5);
        boolean b4 = (10 == 20);

        Main obj = new Main();
    }
}

```



```
        System.out.println("b1 = " + b1);
        System.out.println("b2 = " + b2);
        System.out.println("b3 = " + b3);
        System.out.println("b4 = " + b4);
        System.out.println("Default value = " + obj.defaultValue);
    }
}
```

b1 = true
b2 = false
b3 = true
b4 = false
Default value = false

```
public class Main {

    boolean defaultValue; //instance variable

    public static void main(String[] args) {

        Main obj = new Main();

        System.out.println(obj.defaultValue);
    }
}
```

false

```
public class Main {

    static boolean defaultValue;

    public static void main(String[] args) {

        System.out.println(defaultValue);
    }
}
```

False

Member	Belongs to	Access from main()
boolean defaultValue;	Object (instance)	☐ Need an object
static boolean defaultValue;	Class	☐ Directly accessible

Character Escape Sequences

```

public class EscapeExample
{
    public static void main(String args[])
    {
        String str = new String ("My name is \'abcd\'");
        System.out.println(str);

        str = "My Lab is \"JAVA LAB\"";
        System.out.println(str);

        str = "My work files are in D:\\\\Work Projects\\java";
        System.out.println(str);

        str = "First Line \\nSecond Line";
        System.out.println(str);

        str = "Ravindra\\b\\b" + "College";
        System.out.println(str);

        str = "Ravindra \\rCollege";
        System.out.println(str);

        str = "Ravindra \\fCollege";
        System.out.println(str);
    }
}

```

```

C:\Users\RCEW LAB 1>java EscapeExample
My name is 'abcd'
My Lab is "JAVA LAB"
My work files are in D:\Work Projects\java
First Line
Second Line
RavindraCollege
Collegea
Ravindra fCollege
C:\Users\RCEW LAB 1>

```

Variables

The variable is the basic unit of storage in a Java program.

- A variable is defined by the combination of an identifier, a type, and an optional initializer.

- In addition, all variables have a scope, which defines their visibility, and a lifetime

```
int a, b, c;           // declares three ints, a, b, and c.
int d = 3, e, f = 5;   // declares three more ints, initializing
                      // d and f.
byte z = 22;           // initializes z.
double pi = 3.14159;   // declares an approximation of pi.
char x = 'x';          // the variable x has the value 'x'.
```

```
// Demonstrate dynamic initialization.
```

```
class DynInit {
    public static void main(String args[]) {
        double a = 3.0, b = 4.0;
        // c is dynamically initialized
        double c = Math.sqrt(a * a + b * b);
        System.out.println("Hypotenuse is " + c);
    }
}
```

The Scope and Lifetime of Variables

```
// Demonstrate block scope.
class Scope {
    public static void main(String args[]) {
        int x; // known to all code within main
        x = 10;
        if(x == 10) { // start new scope
            int y = 20; // known only to this block
            // x and y both known here.
            System.out.println("x and y: " + x + " " + y);
            x = y * 2;
        }
        // y = 100; // Error! y not known here
        // x is still known here.
        System.out.println("x is " + x);
    }
}
```

Scanner

- `public String nextLine()` :- Moves the scanner to the next line and returns the skipped input.
- `public String next()` :- Returns the token before delimiter
- `public byte nextByte()` :- Scans next token as byte value
- `public short nextShort()` :- Scans next token as short value
- `public int nextInt()` :- Scans next token as integer value
- `public long nextLong()` :- Scans next token as long value
- `public float nextFloat()` :- Scans next token as float value
- `public double nextDouble()` :- Scans next token as double value
- `void close` :- Scanner is closed

next(): Read One word stops at Space, tab, or newline

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your full name: ");
        String name = sc.next();

        System.out.println("Name = " + name);

        sc.close();
    }
}
```

Input: Ashok Rajan

Output: Name = Ashok

nextLine(): Read the entire line

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter your full name: ");
String a = sc.nextLine();
System.out.println(a);
```

Input: Ashok Rajan R
Output: Ashok Rajan R

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Read byte
        System.out.print("Enter a byte value: ");
        byte b = sc.nextByte();

        // Read short
        System.out.print("Enter a short value: ");
        short s = sc.nextShort();

        // Read int
        System.out.print("Enter an int value: ");
        int i = sc.nextInt();

        // Read long
        System.out.print("Enter a long value: ");
        long l = sc.nextLong();

        // Read float
        System.out.print("Enter a float value: ");
        float f = sc.nextFloat();

        // Read double
        System.out.print("Enter a double value: ");
        double d = sc.nextDouble();

        System.out.println("\n----- Values Entered -----");
        System.out.println("Byte   : " + b);
        System.out.println("Short  : " + s);
        System.out.println("Int    : " + i);
        System.out.println("Long   : " + l);
        System.out.println("Float  : " + f);
        System.out.println("Double : " + d);

        sc.close();
    }
}
```

```
}  
}
```

Enter a byte value: 100
Enter a short value: 20000
Enter an int value: 500000
Enter a long value: 9876543210
Enter a float value: 12.5
Enter a double value: 45.6789

----- Values Entered -----

Byte : 100
Short : 20000
Int : 500000
Long : 9876543210
Float : 12.5
Double : 45.6789

Non-Primitive (Reference) Data Types

Non-primitive data types are derived from primitive types or created by programmers.

One-Dimensional Arrays

```
public class Main {  
    public static void main(String[] args) {  
  
        int[] arr = {10, 20, 30, 40, 50};  
  
        System.out.println("Array Elements:");  
  
        for (int i = 0; i < arr.length; i++) {  
            System.out.println(arr[i]);  
        }  
    }  
}
```

```

class Testarray{
public static void main(String args[]){
    int a[]=new int[5];//declaration and instantiation

    a[0]=10;//initialization
    a[1]=20;
    a[2]=70;
    a[3]=40;
    a[4]=50;

    //traversing array
    for(int i=0;i<a.length;i++)//length is the of array
        System.out.println(a[i]);
}
}

```

Declaration	Valid?
int[] a;	✓ Recommended
int a[];	✓
int[] a, b;	✓ Both arrays
int a[], b[];	✓ Both arrays
int a[], b;	✓ a array, b int
int a[10];	❑ Invalid in Java

Multidimensional Arrays

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
    }
}

```



```

int[][] a = new int[3][3];

// Input
System.out.println("Enter 9 elements:");

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        a[i][j] = sc.nextInt();
    }
}

// Display
System.out.println("\nMatrix:");

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        System.out.print(a[i][j] + " ");
    }
    System.out.println();
}

sc.close();
}

```

Input

10 20 30

40 50 60

70 80 90

Output

Matrix:

10 20 30

40 50 60

70 80 90

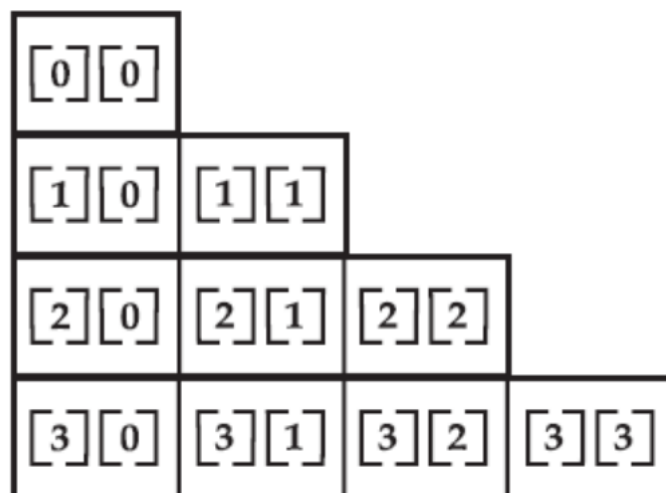
Declaration	Valid?
int[][] a;	✓ Recommended

<code>int a[][];</code>	✓
<code>int[] a[];</code>	✓
<code>int[][] a, b;</code>	✓ Both 2D arrays
<code>int[] a[], b;</code>	✓ a 2D array, b 1D array
<code>int[][] a = new int[3][4];</code>	✓
<code>int a[][] = new int[3][4];</code>	✓

Jagged Array in Java

A Jagged Array (also called a **Ragged Array**) is a 2D array in which each row can have a different number of columns.

- Creating odd number of columns in a 2D array, it is known as a jagged array.
- In other words, it is an array of arrays with **different number of columns**.



```
public class Main {
    public static void main(String[] args) {

        int[][] a = {
            {10, 20, 30},
            {40, 50, 60, 70, 80},
            {90, 100}
        }
```

```

};

for (int i = 0; i < a.length; i++) {
    for (int j = 0; j < a[i].length; j++) {
        System.out.print(a[i][j] + " ");
    }
    System.out.println();
}
}
}
}

```

```

10 20 30
40 50 60 70 80
90 100

```

Creating Jagged Array Using new

```

public class Main {
    public static void main(String[] args) {

        int[][] a = new int[3][];

        a[0] = new int[2];
        a[1] = new int[4];
        a[2] = new int[3];

        a[0][0] = 10;
        a[0][1] = 20;

        a[1][0] = 30;
        a[1][1] = 40;
        a[1][2] = 50;
        a[1][3] = 60;

        a[2][0] = 70;
        a[2][1] = 80;
        a[2][2] = 90;

        for (int i = 0; i < a.length; i++) {
            for (int j = 0; j < a[i].length; j++) {
                System.out.print(a[i][j] + " ");
            }
            System.out.println();
        }
    }
}

```

```

    }
}

10 20
30 40 50 60
70 80 90

```

3D Array

```

// Demonstrate a three-dimensional array.
class ThreeDMatrix {
public static void main(String args[]) {
    int threeD[][][] = new int[3][4][5];
    int i, j, k;
    for(i=0; i<3; i++)
        for(j=0; j<4; j++)
            for(k=0; k<5; k++)
                threeD[i][j][k] = i * j * k;

    for(i=0; i<3; i++) {
        for(j=0; j<4; j++) {
            for(k=0; k<5; k++)
                System.out.print(threeD[i][j][k] + " ");
            System.out.println();
        }
        System.out.println();
    }
}
}

```

```

output:
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0

0 0 0 0 0
0 1 2 3 4
0 2 4 6 8
0 3 6 9 12

0 0 0 0 0
0 2 4 6 8
0 4 8 12 16
0 6 12 18 24

```